IcfaiTech
Faculty of Science & Technology (FST)

SKYROUTE: AIRLINE BOOKING, BAGGAGE & STAFF OPERATIONS SYSTEM

A Mini Project Report Submitted for the Course

Database Management Systems (CS/AI/AIM/DS202)

Submitted by

D.Sivasai Bhaskar Sarma

Roll Number: 25STUCHH010868 Section: D

Instructor-in-Charge: Dr. Meena Kumari Parigi

Faculty of Science & Technology (FST)

IcfaiTech, IFHE Hyderabad

Abstract

Booking a seat at an airline counter still runs on a printed seat chart, a whiteboard for gates, and a calculator for excess baggage. Two agents can sell the same seat within the same minute. Two flights can be given the same gate. The fee depends on who is at the counter, and each station keeps a passenger list that the other station cannot see.

SkyRoute puts the whole operation into one MySQL database: five airports, three airlines, sixteen scheduled flights and 336 dated departures. The design separates a timetable flight from the dated departure it produces, so one schedule row owns as many departures as the calendar needs. Staff sit in a shared employee table, with three role tables hanging off the same key.

The rules that used to sit in people's heads now sit in the database. A unique key stops the double sale. A trigger stops the oversold flight. A check stops the negative price. The baggage fee is a view, so every receipt adds up the same way. A stored procedure books the seat and opens the payment in one transaction.

By the end of the build the database held 34,708 bookings, 43,314 bags and 4,000 passengers, and each rule was tested against that data rather than against three sample rows.

Step 1: Title Selection

The title names the three things the system does: bookings, baggage and staff operations. It avoids "Airport Management System", a phrase that could mean anything from runway maintenance to retail concessions, and it avoids the single-route shuttle framing as well.

The subtitle carried through the report is "one database for every station", which is the point of the project in five words.

Step 2: Problem Statement

The work started from the booking counters at Hyderabad (HYD) and Bengaluru (BLR). Seat charts were printed per flight. Gate changes went on a whiteboard. Baggage fees were worked out by hand, and each station kept its own passenger list.

Five failures follow from that, and each one is closed by a mechanism described later in this report.

- Two agents sell the same seat. Nothing serialises them, so the second sale overwrites the first. Closed by a unique key on (departure_id, seat_number), with the capacity trigger as a second guard.
- Two flights are given the same gate. The clash is written on a whiteboard and nobody checks it. Closed by a unique key on (terminal_id, gate_number), so a physical gate is one row.
- Baggage fees come out differently depending on who is at the counter. Closed by moving the rule into a view: 20 kg free per bag, Rs 500 for each kilo above that.
- Passenger lists disagree between stations. Closed by one database for all five airports, with airports as the root table.
- A flight sells more seats than the aircraft holds. Closed by a before-insert trigger that counts the seats already sold and refuses the next row with SQLSTATE 45000.

Step 3: Objectives

- Schedule flights and allocate gates across five airports from one database.
- Make a double sale and an oversold flight impossible inside the database, whichever application writes the row.
- Compute the baggage fee from weighed kilograms, in the same way on every receipt.
- Answer the departure board, manifest, revenue, load factor and gate utilisation questions with one query each.
- Give the three staff roles only the access they need, and check that against the live grant tables.
- Measure the cost of the four busiest query patterns before and after indexing.
- Carry the same design into a Flask application with working create, read, update and delete.

Step 4: Requirements Gathering

The system adds, edits and views airports, airlines, aircraft, timetable flights and dated departures. It assigns an aircraft and a gate to each departure, sells seats, records bag weights, takes payments, checks passengers in, and manages staff records.

It also has to stay responsive at tens of thousands of rows, keep seat allocation correct when two sellers act at once, and follow least privilege for the three roles.

Three things were left out on purpose. Crew rostering is a scheduling problem, so staff exist as records and daily assignment belongs to the application. Runways are not modelled separately, because gate and terminal already carry the clash story. Fares are stored per booking rather than computed, since revenue is reported and not priced.

The model also assumes one currency (INR), one time zone (IST) and single-leg itineraries. Codeshares are out of scope, so a flight belongs to exactly one airline. These are stated because each one would change the schema if it were relaxed.

Step 5: Entity-Relationship Modeling

Sixteen entities cover the operation: Airport, Airline, Aircraft, Flight, Departure, Terminal, Gate, Passenger, Booking, Baggage, Payment, Check_In, Employees, Pilots, Crew_Members and Ground_Staff. Baggage is drawn as a weak entity, because a bag has no identity outside its booking.

- Airline 1-N Aircraft. A fleet belongs to one airline.
- Flight N-1 Airports, twice: once as departure, once as arrival.
- Flight 1-N Departures. The timetable row is the template, the departure is a date.
- Airport 1-N Terminals, Terminal 1-N Gates.
- Passenger 1-N Bookings N-1 Departures. A booking resolves one passenger onto one departure and carries seat, class and price.
- Booking 1-N Baggage, through an identifying relationship.
- Booking 1-1 Payment, and 1-1 Check_In.
- Employees 1-1 each role table on the shared key, so a licence row cannot exist without its employee.

Keys, cardinality and participation were settled on the diagram before any CREATE TABLE was written. The notation is Chen: rectangles for entities, diamonds for relationships, ovals for

attributes, and double borders for the weak entity and its identifying relationship.

Two relationships carry most of the traffic. A booking sits between one passenger and one departure, and it is where seat, class and price are stored, so the same passenger can fly twice without their details being copied. Baggage hangs off that booking through a weak relationship and is numbered per booking, which is why bag 2 of booking 817 cannot exist once booking 817 is gone.

The diagram was drawn once and then left alone. Every later change to the schema was checked against it first, so the tables follow the model rather than the model being redrawn to match the tables.

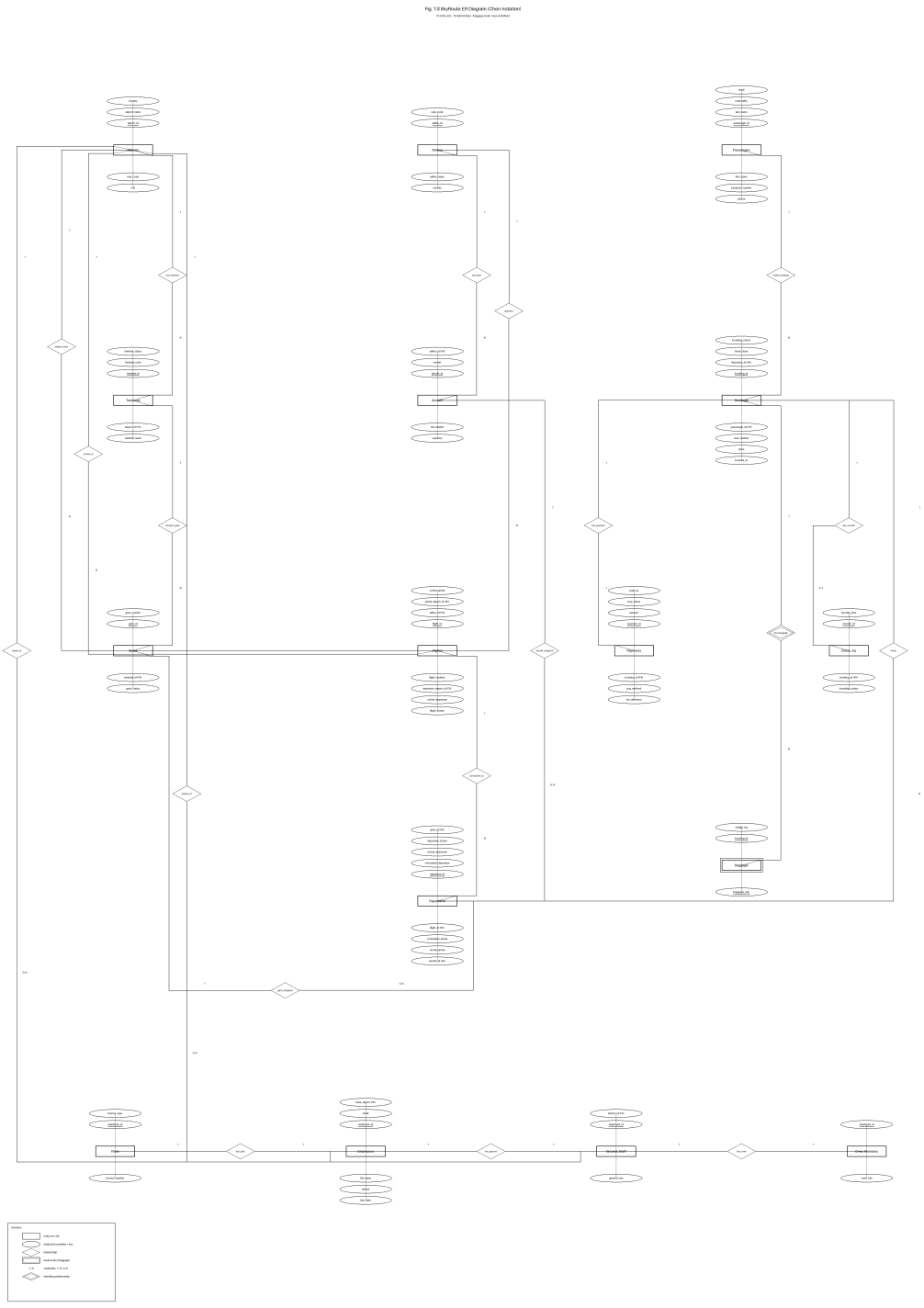


Figure 1: SkyRoute ER diagram (Chen notation, keys, cardinality)

Step 6: Schema Design (Relational Model)

The ER model becomes sixteen tables. Surrogate integer keys keep the foreign keys narrow. Business keys such as IATA codes, tail numbers, emails and transaction references carry UNIQUE constraints instead of serving as primary keys.

Master tables are referenced, never copied, so an airline name appears once rather than in every flight row. That single choice removes most of the update anomalies before they can happen.

Normalisation was checked against one flat booking sheet: passenger name, passport, flight number, departure date, seat, class, price, bag tag, bag weight and fee, all on a single row.

- 1NF: a booking with two bags repeats the tag and the weight, so those columns move to Baggage, keyed by (booking_id, baggage_seq).
- 2NF: passenger details depend on the passenger, not on the pair (passenger, departure), so Passengers splits out.
- 3NF: city depends on the airport, and the airline name depends on the airline, so neither belongs on a booking row.
- BCNF: every determinant that remains is a key, which the unique constraints enforce.

What is left is sixteen tables and nineteen foreign keys, with no repeating group and no transitive dependency.

The rules attached to the tables:

- CASCADE where a child cannot outlive its parent. Bookings, bags and payments disappear with a deleted departure.
- RESTRICT on airports, so a timetable flight is never silently wiped out.
- SET NULL on optional assignments, so removing an aircraft unassigns it rather than deleting history.
- ENUMs for statuses, classes, methods and roles.
- CHECKs for capacity ranges, non-negative prices, weights up to 100 kg, and arrival after departure.

Table 1: The sixteen tables, their primary keys and the rule each one enforces

Table	Primary key	Rule that matters
Airports	airport_id	UNIQUE iata, name
Airlines	airline_id	UNIQUE name, iata
Aircraft	aircraft_id	UNIQUE tail; CHECK cap 1-850
Flights	flight_id	UNIQUE number; dep <> arr airports
Departures	departure_id	UNIQUE(flight, dep datetime)
Terminals / Gates	own ids	UNIQUE per parent (anti-clash)
Passengers	passenger_id	UNIQUE passport, email
Bookings	booking_id	UNIQUE(departure, seat)
Baggage	(booking, seq)	weak entity; 0 < kg <= 100
Payments / Check-Ins	own ids	UNIQUE booking (one-to-one)

Table	Primary key	Rule that matters
Employees + role tables	employee_id	role PK = employee PK

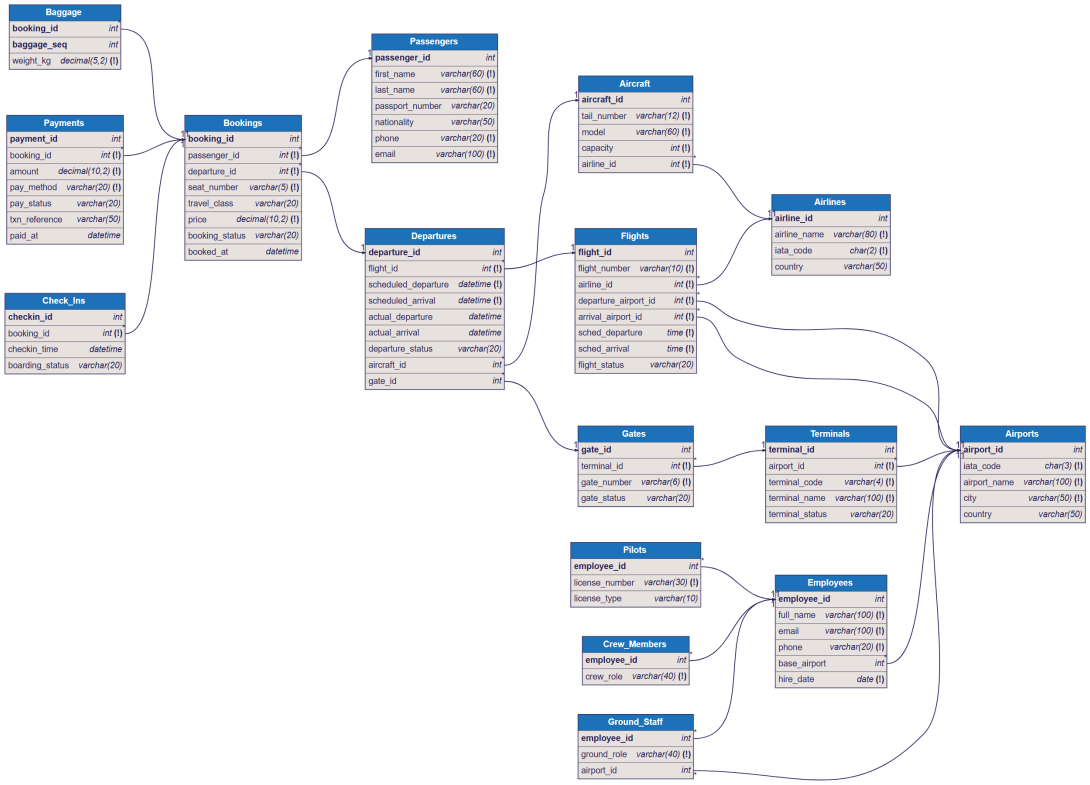


Figure 2: Relational schema of skyroute_db (16 tables, 19 foreign keys)

Step 7: SQL Implementation

The schema was built in two passes. The first pass created the twelve travel tables, the second created the four staff tables and the three database users. Every constraint from the previous step appears in the CREATE statements.

Access control is part of the schema rather than an afterthought. sky_admin has full rights, sky_staff can read and write, and sky_viewer can only read. Each grant is checked with SHOW GRANTS against the exact database name, because a grant on a misspelled name succeeds and protects nothing.

```
mysql> SHOW GRANTS FOR 'sky_admin'@'localhost';
+-------------------------------------------------------------------+
| Grants for sky_admin@localhost                                    |
+-------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `sky_admin`@`localhost`                     |
| GRANT ALL PRIVILEGES ON `skyroute_db`.* TO `sky_admin`@`localhost` |
+-------------------------------------------------------------------+
2 rows in set (0.00 sec)

mysql> SHOW GRANTS FOR 'sky_staff'@'localhost';
+-----------------------------------------------------------------------------+
| Grants for sky_staff@localhost                                              |
+-----------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `sky_staff`@`localhost`                               |
| GRANT SELECT, INSERT, UPDATE ON `skyroute_db`.* TO `sky_staff`@`localhost` |
+-----------------------------------------------------------------------------+
2 rows in set (0.00 sec)

mysql> SHOW GRANTS FOR 'sky_viewer'@'localhost';
+-----------------------------------------------------------+
| Grants for sky_viewer@localhost                           |
+-----------------------------------------------------------+
| GRANT USAGE ON *.* TO `sky_viewer`@`localhost`            |
| GRANT SELECT ON `skyroute_db`.* TO `sky_viewer`@`localhost` |
+-----------------------------------------------------------+
2 rows in set (0.00 sec)
```

Figure 3: Grants for the three roles: admin, staff and read-only viewer

The booking table carries three rules at once: a unique key on the departure and seat pair, a check that the price is not negative, and two foreign keys that point at real passengers and real departures. When the same seat is sold twice, the database refuses the second row and names the constraint that stopped it.

```
mysql> INSERT INTO Bookings (passenger_id, departure_id, seat_number, travel_class, price)
    -> SELECT passenger_id, departure_id, seat_number, travel_class, price
    -> FROM Bookings LIMIT 1;
ERROR 1062 (23000): Duplicate entry '1-20B' for key 'bookings.uq_booking_seat'
```

Figure 4: Duplicate seat rejected by uq_booking_seat (ERROR 1062)

```
  CREATE TABLE Bookings (
booking_id     INT AUTO_INCREMENT PRIMARY KEY,
passenger_id   INT NOT NULL,
departure_id   INT NOT NULL,
seat_number    VARCHAR(5) NOT NULL,
travel_class   ENUM('Economy','Business','First') NOT NULL,
price          DECIMAL(10,2) NOT NULL CHECK (price >= 0),
booking_status ENUM('Confirmed','Cancelled','Refunded') DEFAULT 'Confirmed',
booked_at      DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
UNIQUE KEY uq_booking_seat (departure_id, seat_number),
FOREIGN KEY (passenger_id) REFERENCES Passengers(passenger_id),
FOREIGN KEY (departure_id) REFERENCES Departures(departure_id)
);
GRANT SELECT, INSERT, UPDATE ON skyroute_db.* TO 'sky_staff'@'localhost';
REVOKE INSERT, UPDATE ON skyroute_db.* FROM 'sky_staff'@'localhost';
CREATE INDEX idx_bookings_status ON Bookings (booking_status);
```

The seed was generated with seed 42. It builds five airports, three airlines, sixteen routes, twenty-one days of departures and four thousand passengers. Bookings sit between 35 and 80 per cent load, with seats sampled without replacement.

Payments are 92 per cent settled. Baggage averages about 1.2 bags per booking, split 80/20 around the 20 kg allowance. Check-ins cover seventy per cent of past departures. The seed is checked before loading: no duplicate seat pair, and every foreign key inside its parent range.

One row shows how the pieces fit. Booking 38 is Riya Mehta, seat 10C on 6E-501 from HYD to BLR on 1 November 2026, Business class, Rs 13,879, two bags of 11.23 kg and 7.46 kg, paid by UPI. Nothing in that sentence is stored twice: the passenger is one row in Passengers, the route is one row in Flights, and the bags are two rows in Baggage keyed by (38, 1) and (38, 2).

The Flask application uses a second database that holds the twelve operational tables and none of

the staff tables. Table names, keys and rules are identical. The application simply never touches HR data.

Table 2: What the seed produced, counted with COUNT(*)

Table	Rows	What it holds
airports	5	IATA code, city, country
airlines	3	two-letter code and name
aircraft	16	tail number, model, capacity
flights	16	route, times, base fare
departures	336	dated flight, aircraft, gate
terminals	5	one per airport
gates	20	terminal and gate number
passengers	4,000	passport, name, contact
bookings	34,708	seat, class, price, status
baggage	43,314	tag, weight, booking
payments	34,708	method, amount, status
check_ins	11,544	boarding time and seat
employees	8	shared staff record
pilots	2	licence and flight hours
crew_members	3	cabin role
ground_staff	3	airport and shift

Step 8: Advanced SQL Features

Two views carry the application's read paths. The departure board view joins the timetable to both airport roles, the airline, the aircraft, the gate and the terminal, so the board is one query.

The receipt view returns the price plus the computed baggage fee through a scalar subquery, which keeps it valid under ONLY_FULL_GROUP_BY.

```
  CREATE OR REPLACE VIEW booking_receipt_view AS
SELECT b.booking_id,
CONCAT(p.first_name, ' ', p.last_name) AS passenger,
f.flight_number, d.scheduled_departure,
b.seat_number, b.travel_class, b.price, b.booking_status,
pay.pay_status,
COALESCE(
(SELECT SUM(GREATEST(weight_kg - 20, 0)) * 500
FROM Baggage WHERE booking_id = b.booking_id), 0
) AS baggage_fee_rs
FROM Bookings b
JOIN Passengers p ON b.passenger_id = p.passenger_id
JOIN Departures d ON b.departure_id = d.departure_id
JOIN Flights f    ON d.flight_id = f.flight_id
LEFT JOIN Payments pay ON b.booking_id = pay.booking_id;
```

The join queries answer the questions the desk asks every day. The first joins five tables to print a flight schedule with both airport names, the airline and the aircraft.

```
mysql> SELECT f.flight_number, al.airline_name,
    ->        dep.iata_code AS from_airport, arr.iata_code AS to_airport,
    ->        f.sched_departure, f.sched_arrival, f.flight_status
    -> FROM Flights f
    -> JOIN Airlines al ON f.airline_id = al.airline_id
    -> JOIN Airports dep ON f.departure_airport_id = dep.airport_id
    -> JOIN Airports arr ON f.arrival_airport_id = arr.airport_id
    -> ORDER BY f.flight_number LIMIT 20;
+---------------+--------------+--------------+------------+----------------+---------------+---------------+
| flight_number | airline_name | from_airport | to_airport | sched_departure | sched_arrival | flight_status |
+---------------+--------------+--------------+------------+----------------+---------------+---------------+
| 6E-501        | IndiGo       | HYD          | BLR        | 08:00:00       | 09:20:00      | Active        |
| 6E-502        | IndiGo       | BLR          | HYD        | 10:10:00       | 11:30:00      | Active        |
| 6E-511        | IndiGo       | HYD          | BOM        | 18:05:00       | 19:35:00      | Active        |
| 6E-531        | IndiGo       | HYD          | MAA        | 11:45:00       | 13:00:00      | Active        |
| 6E-541        | IndiGo       | BOM          | HYD        | 07:40:00       | 09:05:00      | Active        |
| 6E-551        | IndiGo       | MAA          | BOM        | 19:10:00       | 21:05:00      | Active        |
| AI-654        | Air India    | DEL          | BOM        | 07:15:00       | 09:25:00      | Active        |
| AI-671        | Air India    | BOM          | MAA        | 13:40:00       | 15:35:00      | Active        |
| AI-682        | Air India    | MAA          | HYD        | 17:20:00       | 18:35:00      | Active        |
| AI-701        | Air India    | HYD          | DEL        | 09:05:00       | 11:15:00      | Active        |
| AI-702        | Air India    | DEL          | BLR        | 12:20:00       | 15:00:00      | Active        |
| SG-893        | SpiceJet     | BLR          | DEL        | 06:30:00       | 09:10:00      | Active        |
| SG-894        | SpiceJet     | DEL          | HYD        | 20:00:00       | 22:15:00      | Active        |
| SG-901        | SpiceJet     | BOM          | DEL        | 09:30:00       | 11:40:00      | Active        |
| SG-902        | SpiceJet     | BLR          | MAA        | 15:25:00       | 16:30:00      | Active        |
| SG-903        | SpiceJet     | DEL          | MAA        | 10:50:00       | 13:45:00      | Active        |
+---------------+--------------+--------------+------------+----------------+---------------+---------------+
16 rows in set (0.00 sec)
```

Figure 5: J1 flight schedule: five-table join across Flights, Airlines and Airports

A LEFT JOIN keeps bookings that have no bags or no check-in, so a missing child row never hides a real booking. The counts come back as zero rather than dropping the row.

```
mysql> SELECT b.booking_id, b.seat_number, b.price,
    ->        pay.pay_status, c.boarding_status,
    ->        COUNT(bg.baggage_seq) AS bags,
    ->        COALESCE(SUM(bg.weight_kg), 0) AS total_bag_kg
    -> FROM Bookings b
    -> LEFT JOIN Payments pay ON b.booking_id = pay.booking_id
    -> LEFT JOIN Check_Ins c  ON b.booking_id = c.booking_id
    -> LEFT JOIN Baggage bg   ON b.booking_id = bg.booking_id
    -> WHERE b.departure_id = 1
    -> GROUP BY b.booking_id
    -> ORDER BY b.seat_number LIMIT 20;
+------------+-------------+----------+------------+-----------------+------+--------------+
| booking_id | seat_number | price    | pay_status | boarding_status | bags | total_bag_kg |
+------------+-------------+----------+------------+-----------------+------+--------------+
|         10 | 10A         |  5374.00 | Paid       | NULL            |    1 |        12.06 |
|         38 | 10C         | 13879.00 | Paid       | NULL            |    2 |        18.69 |
|         97 | 10D         |  4762.00 | Paid       | NULL            |    1 |        10.62 |
|         17 | 10F         |  4427.00 | Paid       | Boarded         |    1 |         7.63 |
|         89 | 11A         |  4616.00 | Paid       | Boarded         |    1 |         8.85 |
|         25 | 11C         |  5264.00 | Paid       | Boarded         |    1 |        16.80 |
|        105 | 11D         |  5128.00 | Paid       | Boarded         |    1 |         7.76 |
|        116 | 11E         |  5164.00 | Pending    | Boarded         |    1 |        12.57 |
|         96 | 12A         |  5396.00 | Paid       | Boarded         |    1 |        23.01 |
|         61 | 12E         |  4977.00 | Paid       | Boarded         |    1 |        16.42 |
|         65 | 13C         |  5097.00 | Paid       | Boarded         |    1 |        10.92 |
|         85 | 13D         |  5086.00 | Paid       | Boarded         |    1 |         8.77 |
|        103 | 13E         |  4269.00 | Paid       | NULL            |    2 |        39.94 |
|         30 | 14A         |  5351.00 | Paid       | Boarded         |    1 |        22.20 |
|         71 | 14C         |  5264.00 | Paid       | Boarded         |    1 |        12.47 |
|         94 | 14D         |  5012.00 | Paid       | NULL            |    1 |        16.02 |
|         47 | 14E         |  4299.00 | Paid       | Boarded         |    1 |        16.50 |
|         44 | 14F         |  4258.00 | Paid       | Boarded         |    2 |        34.99 |
|        125 | 15A         |  5052.00 | Paid       | Boarded         |    2 |        33.53 |
|         51 | 15B         |  5344.00 | Paid       | Boarded         |    1 |        18.74 |
+------------+-------------+----------+------------+-----------------+------+--------------+
20 rows in set (0.02 sec)
```

Figure 6: J4 bookings with payment, check-in and bag totals (LEFT JOIN keeps bag-less rows)

The three staff role tables share the employee key, so one join lists pilots, cabin crew and ground staff in a single directory without a union.

```
mysql> SELECT e.full_name, e.email,
    ->        COALESCE(pl.license_number, cm.crew_role, gs.ground_role) AS credential,
    ->        a.iata_code AS base
    -> FROM Employees e
    -> LEFT JOIN Pilots pl       ON e.employee_id = pl.employee_id
    -> LEFT JOIN Crew_Members cm ON e.employee_id = cm.employee_id
    -> LEFT JOIN Ground_Staff gs ON e.employee_id = gs.employee_id
    -> LEFT JOIN Airports a      ON e.base_airport = a.airport_id
    -> ORDER BY e.full_name;
+--------------+---------------------------+------------------+------+
| full_name    | email                     | credential       | base |
+--------------+---------------------------+------------------+------+
| Amit Verma   | amit.verma@skyroute.aero  | Flight Attendant | HYD  |
| Arjun Mehta  | arjun.mehta@skyroute.aero | Check-in Staff   | HYD  |
| Karan Patel  | karan.patel@skyroute.aero | IND-PIL-2001     | HYD  |
| Meera Das    | meera.das@skyroute.aero   | Baggage Handler  | HYD  |
| Nitin Joshi  | nitin.joshi@skyroute.aero | Flight Attendant | HYD  |
| Pooja Iyer   | pooja.iyer@skyroute.aero  | Ramp Agent       | HYD  |
| Rajesh Gupta | rajesh.gupta@skyroute.aero| Purser           | HYD  |
| Suresh Kumar | suresh.kumar@skyroute.aero| IND-PIL-2002     | HYD  |
+--------------+---------------------------+------------------+------+
8 rows in set (0.01 sec)
```

Figure 7: J10 crew directory across pilots, crew members and ground staff

The analytics side starts with the simplest question: which route sells the most seats. MAA to BOM leads with 2,688 bookings.

```
mysql> SELECT CONCAT(dep.iata_code, ' -> ', arr.iata_code) AS route,
    ->        COUNT(*) AS bookings
    -> FROM Bookings b
    -> JOIN Departures d ON b.departure_id = d.departure_id
    -> JOIN Flights f    ON d.flight_id = f.flight_id
    -> JOIN Airports dep ON f.departure_airport_id = dep.airport_id
    -> JOIN Airports arr ON f.arrival_airport_id = arr.airport_id
    -> GROUP BY route
    -> ORDER BY bookings DESC;
+------------+----------+
| route      | bookings |
+------------+----------+
| MAA -> BOM |     2688 |
| HYD -> BOM |     2664 |
| DEL -> MAA |     2463 |
| DEL -> BLR |     2365 |
| HYD -> BLR |     2358 |
| DEL -> HYD |     2318 |
| HYD -> DEL |     2279 |
| BOM -> DEL |     2157 |
| DEL -> BOM |     2154 |
| BLR -> HYD |     2150 |
| BLR -> DEL |     2142 |
| BOM -> MAA |     2139 |
| BOM -> HYD |     2097 |
| MAA -> HYD |     1991 |
| BLR -> MAA |     1863 |
| HYD -> MAA |      880 |
+------------+----------+
16 rows in set (0.07 sec)
```

Figure 8: Busiest routes: MAA to BOM leads with 2,688 bookings

Load factor is seats sold divided by aircraft capacity, grouped by flight. The HAVING clause keeps only the flights below sixty per cent, which is where the airline would look first.

```
mysql> SELECT f.flight_number,
    ->        COUNT(DISTINCT d.departure_id) AS departures,
    ->        COUNT(b.booking_id) AS seats_sold,
    ->        ROUND(COUNT(b.booking_id) /
    ->              (COUNT(DISTINCT d.departure_id) * MAX(ac.capacity)) * 100, 1) AS load_pct
    -> FROM Flights f
    -> JOIN Departures d ON f.flight_id = d.flight_id
    -> JOIN Aircraft ac ON d.aircraft_id = ac.aircraft_id
    -> LEFT JOIN Bookings b ON d.departure_id = b.departure_id
    -> GROUP BY f.flight_number
    -> HAVING load_pct < 60
    -> ORDER BY load_pct;
+---------------+------------+------------+----------+
| flight_number | departures | seats_sold | load_pct |
+---------------+------------+------------+----------+
| 6E-531        |         21 |        880 |     53.7 |
| SG-893        |         21 |       2142 |     54.0 |
| SG-901        |         21 |       2157 |     54.3 |
| 6E-541        |         21 |       2097 |     55.5 |
| AI-654        |         21 |       2154 |     56.4 |
| AI-682        |         21 |       1991 |     56.4 |
| 6E-502        |         21 |       2150 |     56.9 |
| 6E-511        |         21 |       2664 |     57.1 |
| 6E-551        |         21 |       2688 |     57.7 |
| SG-894        |         21 |       2318 |     58.4 |
| SG-902        |         21 |       1863 |     59.9 |
+---------------+------------+------------+----------+
11 rows in set (0.04 sec)
```

Figure 9: Load factor below 60 per cent (GROUP BY with HAVING)

The baggage fee report sums the excess weight per booking and multiplies by the rate. It finds 8,704 overweight bags and Rs 2,69,74,740 in fees across the seeded data.

```
mysql> SELECT COUNT(*) AS overweight_bags,
    ->        ROUND(SUM((weight_kg - 20) * 500), 2) AS fee_revenue_rs
    -> FROM Baggage
    -> WHERE weight_kg > 20;
+-----------------+-----------------+
| overweight_bags | fee_revenue_rs |
+-----------------+-----------------+
|            8704 |    26974740.00 |
+-----------------+-----------------+
1 row in set (0.01 sec)
```

Figure 10: Overweight baggage: 8,704 bags and Rs 2,69,74,740 in fees

A correlated subquery returns the passengers who spend more than the average fare. It is the one place where a per-row subquery earns its cost, because the average is computed once.

```
mysql> SELECT p.first_name, p.last_name, SUM(b.price) AS total_spent_rs
    -> FROM Bookings b
    -> JOIN Passengers p ON b.passenger_id = p.passenger_id
    -> GROUP BY p.passenger_id
    -> HAVING total_spent_rs > (SELECT AVG(price) FROM Bookings)
    -> ORDER BY total_spent_rs DESC LIMIT 15;
+------------+-----------+----------------+
| first_name | last_name | total_spent_rs |
+------------+-----------+----------------+
| Ishaan     | Verma     |      123886.00 |
| Sneha      | Sharma    |      123521.00 |
| Ishaan     | Das       |      121860.00 |
| Arjun      | Nair      |      119949.00 |
| Aisha      | Rao       |      118430.00 |
| Sneha      | Gupta     |      111366.00 |
| Sanya      | Sharma    |      109966.00 |
| Vikram     | Iyer      |      109152.00 |
| Pooja      | Rao       |      108216.00 |
| Priya      | Verma     |      107405.00 |
| Vikram     | Sharma    |      106833.00 |
| Sneha      | Menon     |      106367.00 |
| Pooja      | Iyer      |      105946.00 |
| Pooja      | Gupta     |      103986.00 |
| Vikram     | Joshi     |      103798.00 |
+------------+-----------+----------------+
15 rows in set (0.10 sec)
```

Figure 11: Passengers spending above the average fare (subquery)

MakeBooking(passenger, departure, seat, class, price, OUT id) writes the booking and opens its Pending payment in one transaction. An EXIT HANDLER rolls back and re-signals on any error, so a taken seat fails the whole call and leaves nothing behind.

```
mysql> SET @d := (SELECT d.departure_id
    ->            FROM Departures d
    ->            JOIN Aircraft a ON d.aircraft_id = a.aircraft_id
    ->            WHERE (SELECT COUNT(*) FROM Bookings b
    ->                   WHERE b.departure_id = d.departure_id) < a.capacity
    ->            ORDER BY d.departure_id LIMIT 1);
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> CALL MakeBooking(1, @d, '99Z', 'Economy', 4500.00, @id);
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> SELECT @id AS new_booking_id;
+----------------+
| new_booking_id |
+----------------+
|          34714 |
+----------------+
1 row in set (0.00 sec)

mysql> SELECT seat_number, travel_class, price FROM Bookings  WHERE booking_id = @id;
+-------------+--------------+---------+
| seat_number | travel_class | price   |
+-------------+--------------+---------+
| 99Z         | Economy      | 4500.00 |
+-------------+--------------+---------+
1 row in set (0.00 sec)

mysql> SELECT pay_status, pay_method         FROM Payments   WHERE booking_id = @id;
+------------+------------+
| pay_status | pay_method |
+------------+------------+
| Pending    | UPI        |
+------------+------------+
1 row in set (0.00 sec)
```

Figure 12: MakeBooking creates booking 34714 and its Pending UPI payment

The capacity trigger fires before every insert on Bookings, whether the row comes from the shell, the procedure or the web application. It counts seats sold against the aircraft capacity and signals SQLSTATE 45000 when the flight is full.

```
 CREATE TRIGGER trg_booking_capacity
BEFORE INSERT ON Bookings
FOR EACH ROW
BEGIN
DECLARE sold INT DEFAULT 0;
DECLARE cap  INT DEFAULT NULL;
SELECT COUNT(*) INTO sold FROM Bookings WHERE departure_id = NEW.departure_id;
SELECT a.capacity INTO cap
FROM Departures d JOIN Aircraft a ON d.aircraft_id = a.aircraft_id
WHERE d.departure_id = NEW.departure_id;
IF cap IS NOT NULL AND sold >= cap THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Booking refused: flight is fully booked';
END IF;
END
```

```
mysql> SET @ac   := (SELECT aircraft_id FROM Departures WHERE departure_id = @d);
Query OK, 0 rows affected (0.00 sec)

mysql> SET @sold := (SELECT COUNT(*) FROM Bookings WHERE departure_id = @d);
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE Aircraft SET capacity = @sold WHERE aircraft_id = @ac;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> INSERT INTO Bookings (passenger_id, departure_id, seat_number, travel_class, price)
    -> VALUES (1, @d, '98Z', 'Economy', 4500.00);
ERROR 1644 (45000): Booking refused: flight is fully booked
```

Figure 13: Capacity trigger refuses an oversold seat (ERROR 45000)

Load factor, revenue, gate utilisation and fee revenue only mean something at volume. On three rows they are arithmetic. On thirty thousand they are the report.

Step 9: Transaction Management

Three transaction scenarios were run against the live database. Each one starts from a known state and ends by restoring it.

- Demo A commits a booking and its payment together. Both rows appear, or neither does.
- Demo B inserts a passenger, then a second passenger with the same passport number. The second insert breaks UNIQUE and the rollback removes the first row. A count afterwards returns zero.

```
mysql> INSERT INTO Passengers (first_name, last_name, passport_number, phone, email)
    -> VALUES ('Demo', 'Duplicate', 'DEMO-PP-1', '9000000001', 'demo.dup@example.com');
ERROR 1062 (23000): Duplicate entry 'DEMO-PP-1' for key 'passengers.uq_passenger_passport'
mysql>
mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT COUNT(*) AS rows_left_behind
    -> FROM Passengers WHERE passport_number = 'DEMO-PP-1';
+------------------+
| rows_left_behind |
+------------------+
|                0 |
+------------------+
1 row in set (0.00 sec)
```

Figure 14: ROLLBACK discards the uncommitted passenger row

- Demo C needs two sessions. The first holds a row with SELECT ... FOR UPDATE, and the second's UPDATE waits. When the first rolls back, the second completes, which proves the lock was real.

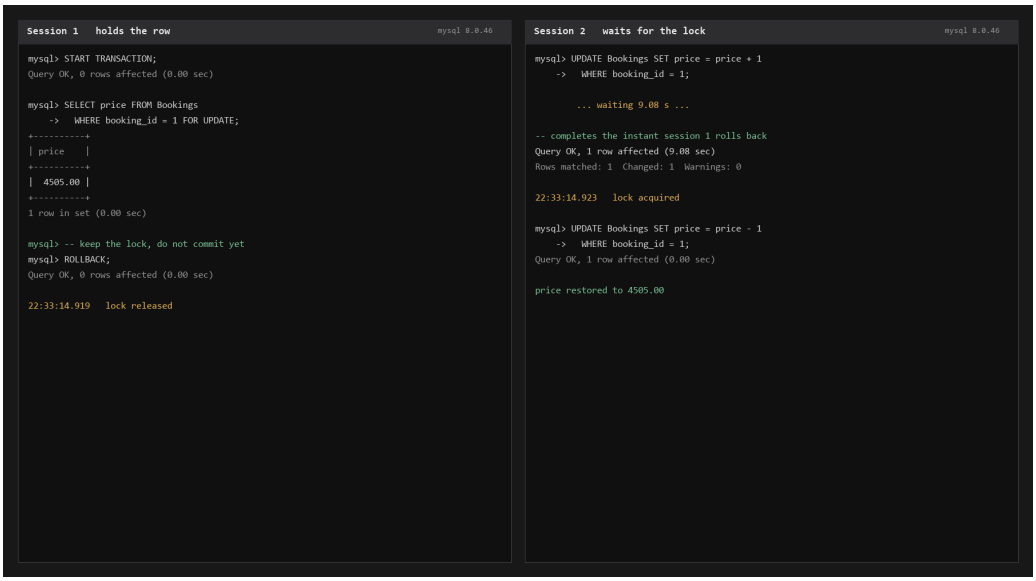


Figure 15: Row lock: session 2 waits 9.08 s until session 1 rolls back (captured console output)

Reading the row again before commit returns the old value. That is isolation, not a definition of it. The second session sees the old value until the first one commits or rolls back.

Step 10: Indexing and Optimization

MySQL already indexes single-column foreign keys, so those were left alone. Four indexes were added for four observed workloads.

- booking_status, because every operations query filters on refunds and pending payments.
- scheduled_departure, because the departure board searches by date range.
- (last_name, first_name), because desk staff look passengers up by family name.
- weight_kg, because the fee report is a range over twenty kilograms.

Each one is tested the same way: EXPLAIN before the index, read the access type and row estimate, create the index, then EXPLAIN again.

The booking-status pair is the clearest. The query uses the index over 5,449 rows, falls to a full scan of 33,904 rows when the index is dropped, and returns to the index once it is rebuilt.

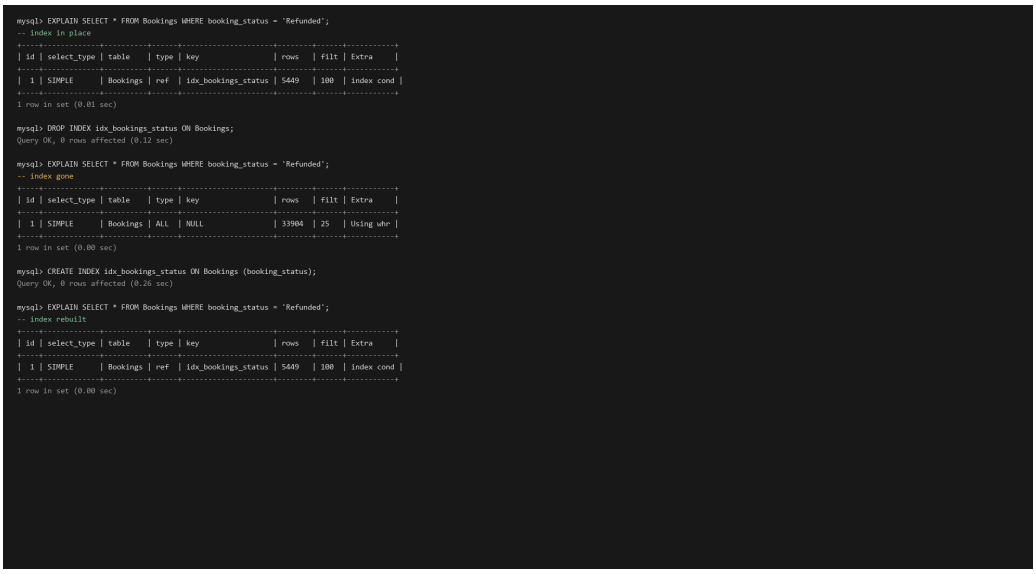


Figure 16: Index before and after: 5,449 rows by index becomes a 33,904-row scan, then back

The other three indexes follow the same pattern, and each one is listed by SHOW INDEX with the columns in the order the query needs them.

Step 11: Testing and Validation

Twenty-four checks were run in eight groups. Each check states its expected result before it runs, so the log works as a protocol as well as evidence.

- Joins (J1-J10): ten clean result sets; the LEFT JOIN keeps bookings with no bags; the overweight list is non-empty by construction.
- Analytics (A1-A8): routes match the scheduled frequencies; every load factor sits between 35 and 80 per cent; fee revenue is positive.
- Constraints (C-a to C-e): a duplicate seat gives ERROR 1062, a same-airport flight gives ERROR 1644, a zero-weight bag is refused, deleting an airport gives ERROR 1451, and deleting a booking removes its bag, payment and check-in.
- Procedure and trigger (T-a to T-c): MakeBooking returns an id with a Pending payment, a

second call on the same seat inserts nothing, and a booking past capacity raises ERROR 45000.

- Transactions (D-A to D-C): commit keeps both rows, rollback frees the seat, and a second session sees the old value.
- Indexes (I-a, I-b): EXPLAIN pairs captured, and SHOW INDEX lists exactly the four new indexes.
- Access control (R-a to R-c): the three grants match the three roles, the viewer is refused an INSERT, and grant names are character exact.
- Scale (S-a, S-b): 336 departures with bookings and bags in the tens of thousands, and the daily snapshot covers all twenty-one days.

The constraint checks are the ones that matter most, because they prove the database refuses bad data rather than merely storing whatever it is given. The row counts afterwards show that each refusal left the data exactly as it was.

```
mysql> UPDATE Aircraft SET capacity = 180 WHERE aircraft_id = @ac;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> DELETE FROM Bookings WHERE booking_id = @id;
Query OK, 1 row affected (0.01 sec)

mysql> SELECT COUNT(*) AS total_bookings FROM Bookings;
+----------------+
| total_bookings |
+----------------+
|          34708 |
+----------------+
1 row in set (0.01 sec)
```

Figure 17: Cleanup restores the aircraft capacity and the count to 34,708

At the end of the run the six largest tables hold the volumes the generator promised: 336 departures, 34,708 bookings, 43,314 bags and 4,000 passengers.

```
mysql> SELECT 'Bookings'   AS table_name, COUNT(*) AS row_count FROM Bookings
    -> UNION ALL SELECT 'Passengers', COUNT(*) FROM Passengers
    -> UNION ALL SELECT 'Departures', COUNT(*) FROM Departures
    -> UNION ALL SELECT 'Baggage',    COUNT(*) FROM Baggage
    -> UNION ALL SELECT 'Flights',    COUNT(*) FROM Flights
    -> UNION ALL SELECT 'Payments',   COUNT(*) FROM Payments;
+------------+-----------+
| table_name | row_count |
+------------+-----------+
| Bookings   |     34708 |
| Passengers |      4000 |
| Departures |       336 |
| Baggage    |     43314 |
| Flights    |        16 |
| Payments   |     34708 |
+------------+-----------+
6 rows in set (0.03 sec)
```

Figure 18: Row counts across the six largest tables

The checks stop where the application begins. Two people booking at the same instant are covered only by the row-lock demo, and restore from backup is not tested at all. Both would need a second host and a second database to do properly.

Step 12: Documentation

This report follows the thirteen project steps in order. Each step says what was built and, where a choice mattered, why it beat the obvious alternative.

The ER diagram and the schema diagram are reproduced in full rather than cropped, and every query result used as evidence is a screenshot with a caption naming the point it proves.

Step 13: Presentation and Reporting

The deck runs to eleven slides: the problem, the mechanism that answers each failure, the ER

model, the schema, the data at volume, the advanced SQL, the transactions, the index measurements, access control and the Flask screen. Each mechanism slide names the check that proves it, so the deck cannot claim more than the tests show.

The demonstration keeps the same order as the report. It opens the departure board, books a seat through the procedure, attempts the same seat a second time to show the refusal, drops an index to show the plan change, and finishes on a receipt with the fee worked out.

Future work

- Partition Bookings by departure month once the table passes a few million rows, so the board query reads one partition instead of the whole table.
- Replace the capacity trigger with a seat-counter row per departure, updated in the same transaction as the booking.
- Move the fee rule into a stored function, so the receipt view and the application call the same code.
- Give the staff tables their own schema and user, so HR data never shares a grant with the travel tables.

Each item is additive. None of them requires the booking, baggage or payment tables to be rebuilt.

Conclusion

Each item in the problem statement is answered by a mechanism that can be pointed at, and each mechanism has a test with a stated expectation behind it. The database holds production-shaped volume while answering board, manifest, revenue and load questions in single queries.

The Flask application reuses the same tables through mysql-connector-python: a staff portal with PIN-gated manifests, check-ins and edits, plus a traveller login restricted to their own bookings. Both screens go through the same views and the same procedure, so a change made at the desk appears on the traveller page without a second code path.

Two limits are worth stating. Bookings are not partitioned by date, so the table will need range partitioning around the point it passes a few million rows. The capacity trigger also counts rows on every insert, which is fine at 34,708 bookings but would become a seat inventory counter under heavier load.

At 34,708 bookings the difference between a good plan and a bad one shows up in milliseconds rather than in correctness. The query returns the same rows either way. What changes is how many rows the engine has to read to find them, and that is the whole point of the index measurements in Step 10.